Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Л и ОА в ИЗ»
на тему «Оценка времени выполнения программ»

Выполнили:
студент группы 25ВВВ1
Кустов И.Д.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., старший преподаватель Деев М.В.

Пенза 2026

Лабораторное задание:

Задание 1:

1. Вычислить порядок сложности программы (*O*-символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки time.h для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции qsort, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание метода решения задачи:

Программа состоит из двух функций: task1_matrix_multiplication() выполняет задание 1, task2_sorting() – задание 2; обе вызываются по очереди из main(). Время измеряется функцией clock() библиотеки time.h: значение счётчика тактов берётся до и после измеряемого участка, разность делится на константу CLOCKS_PER_SEC и даёт время в секундах.

Задание 1:

Матрицы хранятся в одномерных динамических массивах размером N·N (элемент [i][j] имеет индекс i·N + j). Так матрицы размером 4000 и 10000 не переполняют стек. Матрицы a и b заполняются случайными числами от 1 до 100, затем замеряется время только самого перемножения.

Оценка сложности

Перемножение состоит из трёх вложенных циклов по i, j и r, каждый выполняет N итераций. Во внутреннем цикле – одно умножение и одно сложение, поэтому общее число операций равно $N \cdot N \cdot N = N^3$, а сложность алгоритма – $O(N^3)$.

Заполнение матриц требует $O(N^2)$ операций, выделение и освобождение памяти – O(1). Эти слагаемые растут медленнее, чем N^3, поэтому сложность всей программы тоже $O(N^3)$. Отсюда следует, что при увеличении N в 2 раза время должно вырасти примерно в $2^3 = 8$ раз, а при увеличении в 2,5 раза – примерно в $2{,}5^3 \approx 15{,}6$ раза.

Псевдокод

```
sizes = {100, 200, 400, 1000, 2000, 4000, 10000}
ДЛЯ каждого N ИЗ sizes
    a = malloc(N*N*sizeof(int));  b = ...;  c = ...
    ЕСЛИ память не выделена ТО
        вывести "Memory allocation failed", перейти к следующему N
    ДЛЯ i ОТ 0 ДО N*N-1
        a[i] = rand() % 100 + 1
        b[i] = rand() % 100 + 1
    start = clock()
    ДЛЯ i ОТ 0 ДО N-1                          // N раз
        ДЛЯ j ОТ 0 ДО N-1                      // N раз
            elem_c = 0
            ДЛЯ r ОТ 0 ДО N-1                  // N раз  =>  N^3
                elem_c = elem_c + a[i*N + r] * b[r*N + j]
            c[i*N + j] = elem_c
    end = clock()
    вывести N, (end - start) / CLOCKS_PER_SEC
    free(a);  free(b);  free(c)
```

<u>Задание 2:</u>

Сортируются массивы из 50 000 и 100 000 элементов, чтобы сравнить, как время работы алгоритмов растёт при увеличении размера вдвое. Функция generate_array() формирует один из четырёх наборов данных: случайный, возрастающий, убывающий, а также возрастающий в первой половине и убывающий во второй. Перед каждой сортировкой исходный массив копируется функцией memcpy() в рабочий, поэтому все три алгоритма получают одинаковые данные. Для стандартной функции qsort() написана функция сравнения compare_ints().

Псевдокод формирования данных

```
generate_array(arr, n, type)
    ДЛЯ i ОТ 0 ДО n-1
        ВЫБОР type:
            0 → arr[i] = rand()                    // случайный
            1 → arr[i] = i                         // возрастающий
            2 → arr[i] = n - i                     // убывающий
            3 → ЕСЛИ i < n/2 ТО arr[i] = i         // возрастание,
                ИНАЧЕ arr[i] = n - i               // затем убывание
```

Сортировка Шелла

Массив сортируется вставками с уменьшающимся шагом gap = 9, 5, 3, 2, 1. Последний проход (gap = 1) – это обычная сортировка вставками по почти упорядоченному массиву. Для уже отсортированного массива внутренний цикл не выполняется ни разу, и время равно O(n). С

фиксированным набором из пяти шагов в среднем и худшем случае сложность остаётся $O(n^2)$; шаги лишь уменьшают константу.

```
shell(items, count)
    gaps = {9, 5, 3, 2, 1}
    ДЛЯ k ОТ 0 ДО 4
        gap = gaps[k]
        ДЛЯ i ОТ gap ДО count-1
            x = items[i];  j = i - gap
            ПОКА j >= 0 И x < items[j]
                items[j + gap] = items[j]          // сдвиг вправо
                j = j - gap
            items[j + gap] = x
```

Быстрая сортировка

Опорный элемент (компаранд) – средний элемент отрезка. Элементы меньше него переставляются влево, больше – вправо, затем обе части сортируются рекурсивно. Средняя сложность $O(n \cdot \log n)$, худшая $O(n^2)$. Опорный элемент берётся из середины, поэтому на возрастающем и убывающем массивах отрезок делится пополам и худший случай не возникает. Однако на массиве «возрастание, затем убывание» средний элемент оказывается максимальным: за один проход отрезок уменьшается всего на один элемент, время становится $O(n^2)$, а глубина рекурсии – порядка n.

```
qs(items, left, right)
    ЕСЛИ left >= right ТО выход
    i = left;  j = right
    x = items[(left + right) / 2]                  // компаранд
    ПОВТОРЯТЬ
        ПОКА items[i] < x И i < right:  i = i + 1
        ПОКА x < items[j] И j > left:   j = j - 1
        ЕСЛИ i <= j ТО
            обменять items[i] и items[j]
            i = i + 1;  j = j - 1
    ПОКА i <= j
    ЕСЛИ left < j  ТО qs(items, left, j)
    ЕСЛИ i < right ТО qs(items, i, right)
```

Замер времени

```
task2_sorting()
    sizes = {50000, 100000}
    original = malloc(100000);  test_arr = malloc(100000)
    ДЛЯ каждого N ИЗ sizes
        ДЛЯ type ОТ 0 ДО 3
            generate_array(original, N, type)
            ДЛЯ каждого алгоритма ИЗ {shell, qs, qsort}
                memcpy(test_arr, original, N*sizeof(int))
                start = clock()
                выполнить сортировку test_arr
                end = clock()
                вывести название, (end - start) / CLOCKS_PER_SEC
    free(original);  free(test_arr)
```

Результаты работы программы:

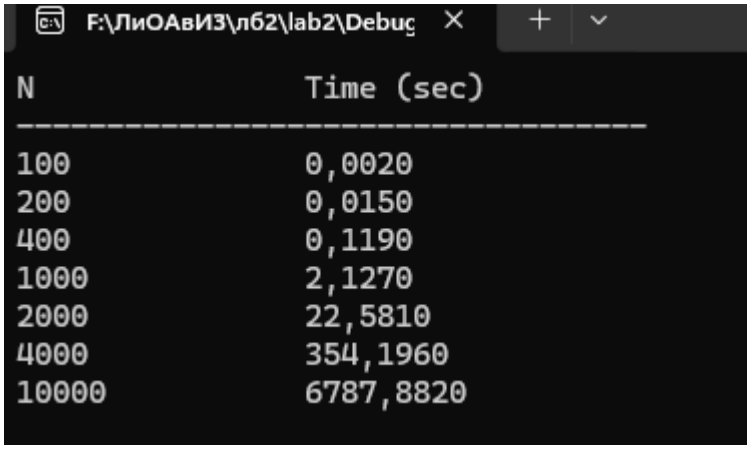


Рисунок 1 – Время перемножения матриц разного размера (задание 1)

На рисунке 1 приведено время перемножения матриц размером от 100 до 10000. Для малых размеров рост близок к теоретическому: при переходе 100 → 200 время выросло в 7,5 раза, 200 → 400 – в 7,9 раза (теория – 8 раз). Для больших матриц рост сильнее теоретического: 1000 → 2000 – в 10,6 раза (теория 8), 2000 → 4000 – в 15,7 раза (теория 8), 4000 → 10000 – в 19,2 раза (теория 15,6). Перемножение матриц 10000 × 10000 заняло 6788 с – около 1 ч 53 мин.

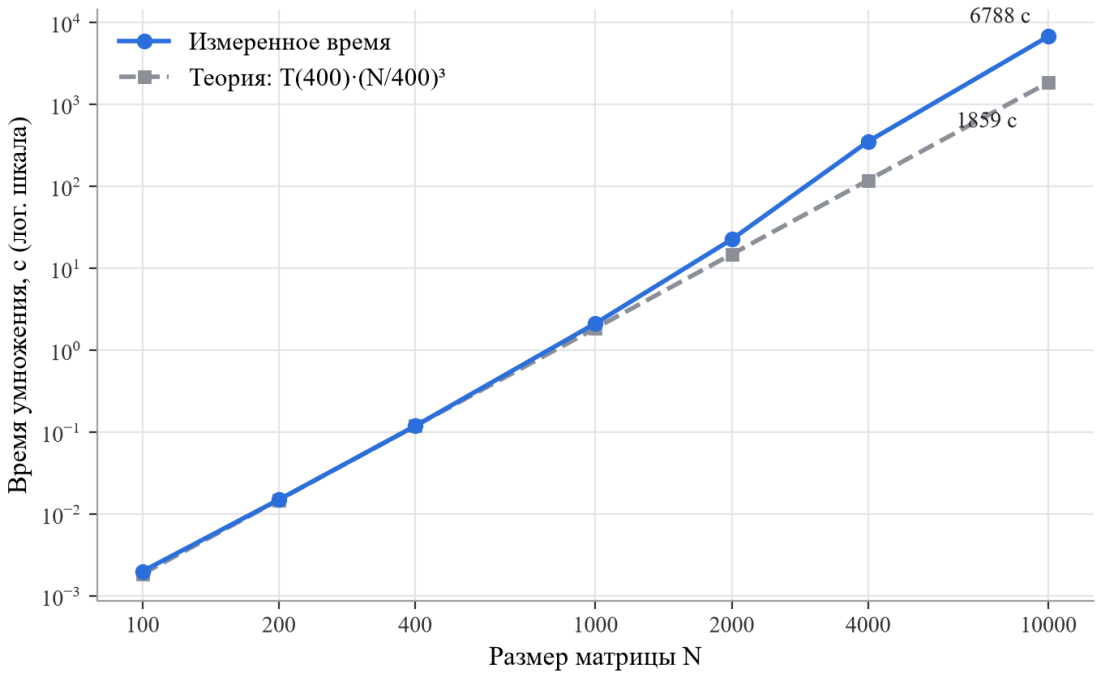


Рисунок 2 – Зависимость времени перемножения от размера матриц

На рисунке 2 сравниваются измеренное время и теоретическая кривая $O(N^3)$. Теоретическая кривая построена по формуле $T(N) = T(400) \cdot (N / 400)^3$, где $T(400) = 0{,}119$ с – измеренное время для $N = 400$. Обе оси логарифмические, поэтому зависимость вида N^3 выглядит прямой линией.

До $N = 1000$ измерения совпадают с теорией (расхождение не более 14 %). С $N = 2000$ реальное время растёт быстрее: при $N = 4000$ оно в 3 раза

больше теоретического, при N = 10000 – в 3,6 раза (6788 с против 1859 с). Причина в работе кэш-памяти процессора. Матрица b читается по столбцам (b[r·N + j]), и соседние обращения отстоят друг от друга на N элементов. Пока матрицы маленькие, они целиком помещаются в кэш. При больших N почти каждое обращение к b вызывает кэш-промах и чтение из оперативной памяти, поэтому одна операция обходится дороже. Порядок сложности при этом остаётся $O(N^3)$, растёт только константа.

```
=== ЗАДАНИЕ 2: Алгоритмы сортировки ===

--- Набор данных: Random (Случайный) (N=50000) ---
Shell Sort:     0,1390 sec
Quick Sort:     0,0050 sec
Std qsort:      0,0080 sec

--- Набор данных: Ascending (Возр.) (N=50000) ---
Shell Sort:     0,0000 sec
Quick Sort:     0,0010 sec
Std qsort:      0,0030 sec

--- Набор данных: Descending (Убыв.) (N=50000) ---
Shell Sort:     0,2730 sec
Quick Sort:     0,0010 sec
Std qsort:      0,0030 sec

--- Набор данных: Asc-Desc (Возр->Убыв) (N=50000) ---
Shell Sort:     0,1360 sec
Quick Sort:     0,6000 sec
Std qsort:      0,0120 sec
```

Рисунок 3 – Время работы алгоритмов сортировки, N = 50 000 (задание 2)

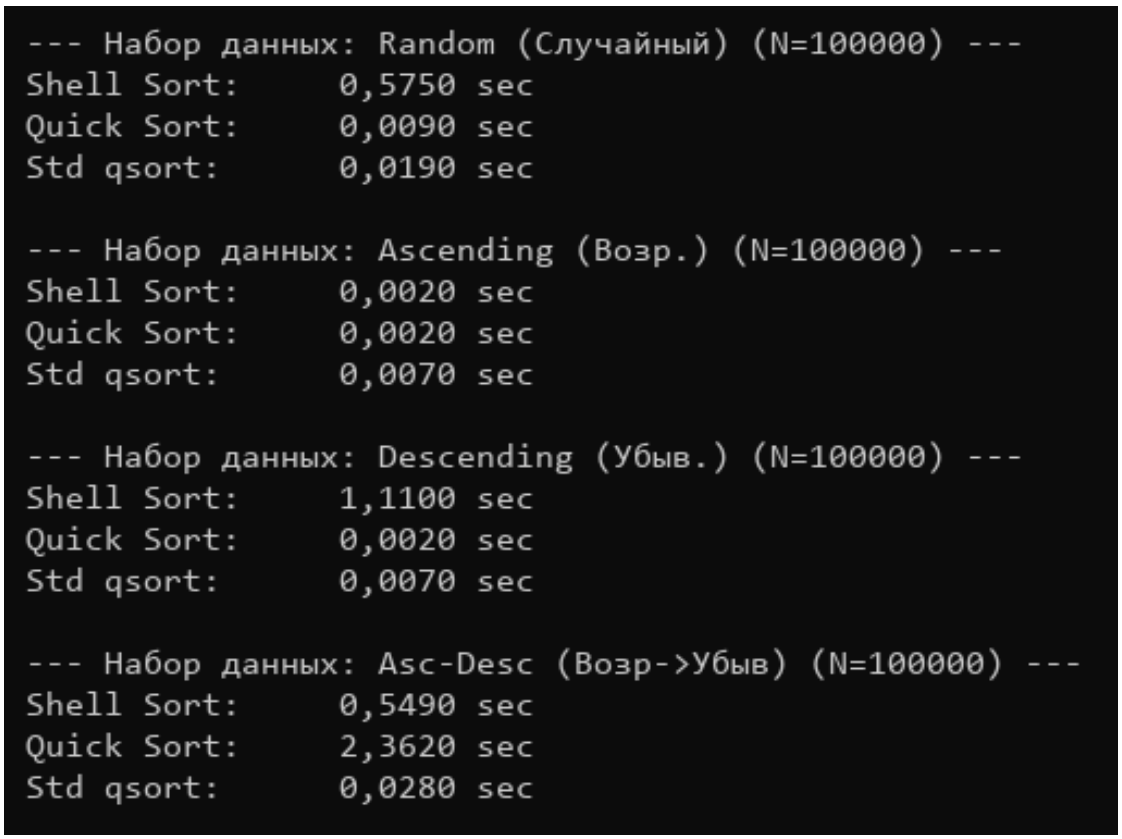


Рисунок 4 – Время работы алгоритмов сортировки, N = 100 000 (задание 2)

На рисунках 3 и 4 приведено время сортировки массивов из 50 000 и 100 000 элементов. Оба замера выполнены одним запуском программы.

Случайный набор: при N = 100 000 сортировка Шелла заняла 0,575 с, быстрая сортировка – 0,009 с, qsort – 0,019 с. Сортировка Шелла с пятью фиксированными шагами работает за $O(n^2)$ и примерно в 60 раз медленнее быстрой сортировки, сложность которой $O(n \cdot \log n)$.

Возрастающий набор: при N = 100 000 Шелл – 0,002 с, быстрая сортировка – 0,002 с, qsort – 0,007 с. Массив уже отсортирован, поэтому сортировка Шелла не сдвигает ни одного элемента и проходит массив за $O(n)$ – это её лучший случай.

Убывающий набор: при N = 100 000 Шелл – 1,110 с, быстрая сортировка – 0,002 с, qsort – 0,007 с. Для сортировки Шелла это худший случай: каждый элемент приходится сдвигать на максимальное расстояние, и время вдвое больше, чем на случайных данных. Быстрая сортировка с опорным элементом из середины делит убывающий массив ровно пополам, поэтому работает так же быстро, как на возрастающем.

Набор «возрастание, затем убывание»: при N = 100 000 Шелл – 0,549 с, быстрая сортировка – 2,362 с, qsort – 0,028 с. Это худший случай для быстрой сортировки: в середине такого массива стоит его максимальный элемент, и именно он выбирается компарандом. Разбиение отделяет от отрезка лишь один элемент, число сравнений растёт как n^2, и сортировка работает примерно в 260 раз дольше, чем на случайных данных.

Глубина рекурсии в этом случае достигает десятков тысяч вызовов. При стандартном размере стека 1 МБ программа аварийно завершалась с кодом 0xC00000FD (переполнение стека) сразу после замера сортировки Шелла. Поэтому в свойствах проекта (Компоновщик → Система → Размер резерва стека) стек увеличен до 256 МБ, после чего замер выполнился полностью.

Стандартная функция qsort() на случайных и упорядоченных наборах медленнее собственной реализации быстрой сортировки. Она универсальна: каждое сравнение выполняется вызовом функции compare_ints() через указатель, а элементы переставляются побайтно с размером, переданным параметром. Зато qsort() выбирает опорный элемент по нескольким элементам отрезка, поэтому на наборе «возрастание, затем убывание» не деградирует. Её время почти не зависит от исходного порядка данных (0,007–0,028 с при N = 100 000).

Сравнение результатов для 50 000 и 100 000 элементов

Таблица 1 – Время сортировки при N = 50 000 и N = 100 000

Набор данных	**Алгоритм**	**N = 50 000, с**	**N = 100 000, с**	**Рост, раз**
Случайный	Шелл	0,139	0,575	4,1
	Быстрая (qs)	0,005	0,009	1,8
	qsort	0,008	0,019	2,4
Возрастающий	Шелл	0,000	0,002	–
	Быстрая (qs)	0,001	0,002	–
	qsort	0,003	0,007	–
Убывающий	Шелл	0,273	1,110	4,1
	Быстрая (qs)	0,001	0,002	–
	qsort	0,003	0,007	–
Возрастание, затем убывание	Шелл	0,136	0,549	4,0
	Быстрая (qs)	0,600	2,362	3,9
	qsort	0,012	0,028	2,3

При увеличении размера массива вдвое время алгоритма со сложностью $O(n^2)$ должно вырасти в $2^2 = 4$ раза, а алгоритма со сложностью $O(n \cdot \log n)$ – в $2 \cdot \log_2 100000 / \log_2 50000 \approx 2{,}1$ раза.

Сортировка Шелла на случайном, убывающем и смешанном наборах замедлилась в 4,0–4,1 раза, что подтверждает квадратичную сложность. Быстрая сортировка на наборе «возрастание, затем убывание» замедлилась в 3,9 раза – это тоже квадратичный рост, характерный для её худшего случая. На случайном наборе быстрая сортировка и qsort замедлились в 1,8 и 2,4 раза, что близко к теоретическим 2,1 раза для $O(n \cdot \log n)$.

Для времён порядка нескольких миллисекунд отношение не вычислялось: точность функции clock() составляет около 1 мс, и погрешность замера сравнима с самим значением.